Estruturas de Dados em Python

O Guia Definitivo: Listas, Tuplas e Dicionários para Lógica de Programação

Prof. Daniel T.

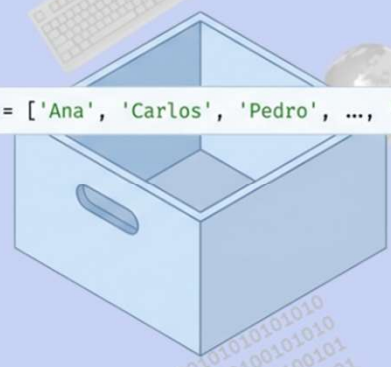
O Problema dos Dados Soltos

```
aluno1 = 'Ana'
aluno2 = 'Carlos'
aluno3 = 'Pedro'
aluno4 = 'José'
aluno5 = 'Beatriz'
aluno6 = 'Ricardo'
aluno7 = 'Mariana'
aluno8 = 'Felipe'
aluno9 = 'Sofia'
aluno10 = 'Lucas'
aluno11 = 'Julia'
aluno12 = 'Gabriel'
aluno13 = 'Isabella'
aluno14 = 'Daniel'
aluno15 = 'Camila'
aluno15 = 'Camila'
aluno16 = 'Miguel'
aluno17 = 'Manuela'
aluno18 = 'Arthur'
aluno19 = 'Eduarda'
aluno20 = 'Matheus'
aluno20 = 'Mora'
aluno100 = 'Maria'
```

⚠ 100 variáveis soltas. Impossível organizar ou pesquisar.

A Solução com Estruturas

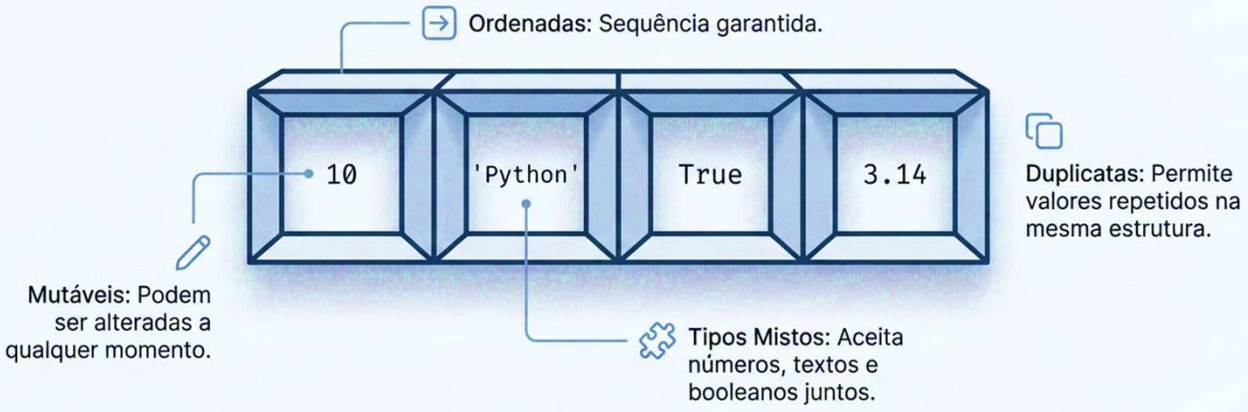
```
alunos = ['Ana', 'Carlos', 'Pedro', …, 'Maria']
```



✅ 1 variável. Dados organizados, rápidos e eficientes.

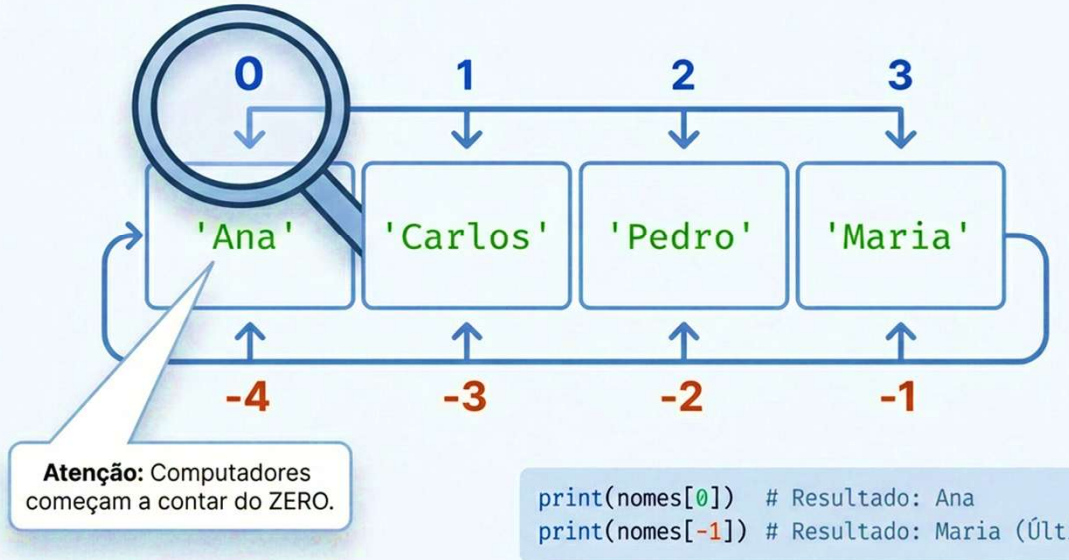
Anatomia de uma Lista

```
misturada = [10, 'Python', True, 3.14]
```



NotebookLM

O Segredo do Índice Zero




```
print(nomes[0])  # Resultado: Ana
print(nomes[-1]) # Resultado: Maria (Último item)
```

NotebookLM

Mutabilidade: Alterando a Matriz

Listas são mutáveis. Você pode reescrever o valor de qualquer posição usando o sinal de atribuição = sem criar uma nova variável.

NotebookLM

Como Percorrer uma Lista (Loops)

Acesso Direto (O Ímã)

```
for nome in nomes:
    print(nome)
```

Pega o dado diretamente. Use quando não se importar com a posição exata do item.

Acesso por Índice (A Lupa)

```
for i in range(len(nomes)):
    print(i, nomes[i])
```

Percorre as numerações (0, 1, 2...). Use quando precisar do índice e do valor ao mesmo tempo.

NotebookLM

O Motor de Automação: Percorrendo com for

O laço “for” visita automaticamente cada “gaveta” da lista, da primeira à última, extraindo seu valor para a variável temporária.

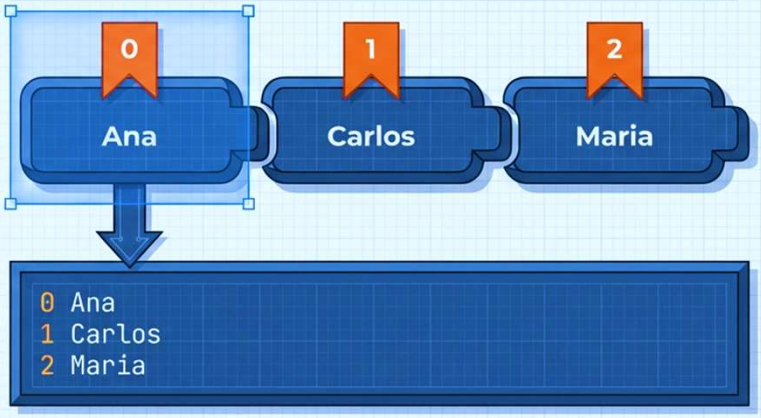
```
nomes = ["Ana", "Carlos", "Maria"]

for nome in nomes:
    print(nome)
```

NotebookLM

Automação Avançada: Percorrendo por Índices

Use este método quando precisar da posição exata do elemento além do seu valor. A função len() mede o tamanho, e range() cria a trilha numérica.

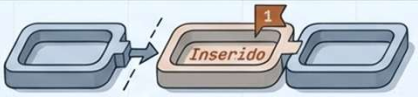
```
nomes = [
    "Ana", "Carlos", "Maria"
]
for i in range(len(nomes)):
    print(i, nomes[i])
```



NotebookLM

Matriz de Diagnóstico: O Cinto de Utilidades

Adicionar e remover elementos requer a ferramenta certa. Entenda a diferença entre operar por Índice vs. operar por Valor.

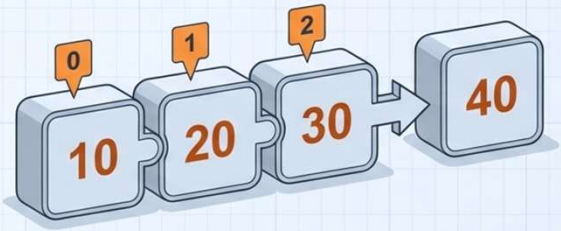
Método	Alvo (Como opera)	Ação Visual
`append()`	Adiciona ao final	
`insert()`	Adiciona via Índice	
`remove()`	Remove pelo VALOR	
`pop()`	Remove pelo ÍNDICE	

NotebookLM

Expandindo a Estrutura (Inserção)

Método: **append()**

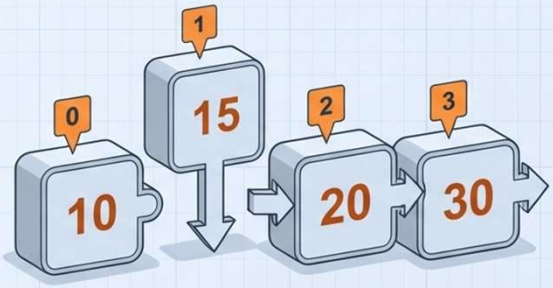
```
numeros = [10, 20, 30]
numeros.append(40)
```



Resultado: `[10, 20, 30, 40]`

Método: **insert()**

```
numeros = [10, 20, 30]
numeros.insert(1, 15)
```



Resultado: `[10, 15, 20, 30]`

NotebookLM

Reduzindo a Estrutura (Remoção)

remove() - Caça ao Valor

```
numeros = [10, 20, 30]

numeros.remove(20)
```

Resultado: [10, 30]

pop() - Caça ao Índice

```
numeros = [10, 20, 30]

numeros.pop(1)
```

Resultado: [10, 30]

Nota: numeros.pop() sem índice remove o último elemento.

NotebookLM

O Canivete Suíço das Listas

num = [10, 20, 30]

`len(num)`

Retorna o tamanho da lista (3).

`sum(num)`

Retorna a soma total dos valores (60).

`max(num)`

Encontra o maior valor presente (30).

`min(num)`

Encontra o menor valor presente (10).

`num.pop()`

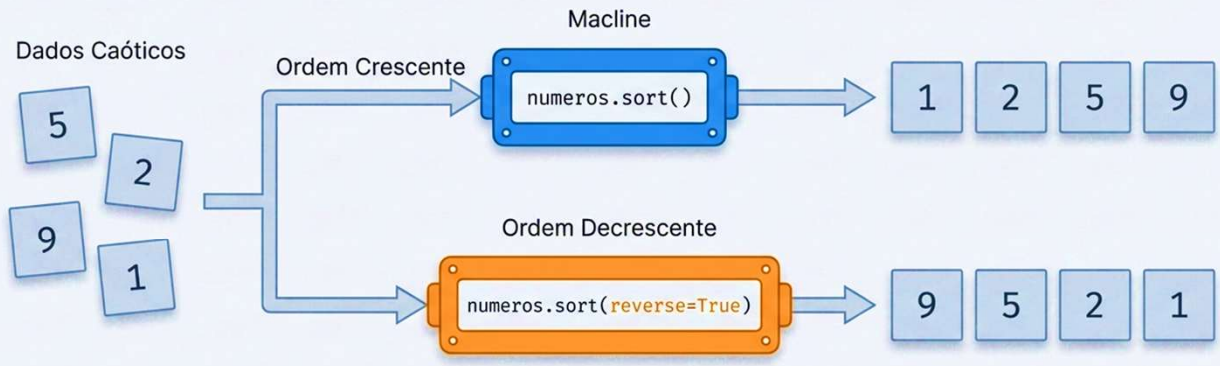
Remove e retorna o último elemento da lista.

`num.pop(1)`

Remove um elemento pelo índice exato (remove o 20).

NotebookLM

Ordenação de Dados



Dica: Quer apenas virar a lista de trás para frente sem ordenar os valores? Use `numeros.reverse()`

NotebookLM

Tuplas: A Lista Blindada

`lista = [10, 20, 30]`
Mutável (Aberto)

`tupla = (10, 20, 30)`
Imutável (Blindada)

`tupla[1] = 50 #` ❌ ERRO!
Uma vez criadas, as tuplas não podem ser alteradas, adicionadas ou removidas.

Hack de Conversão

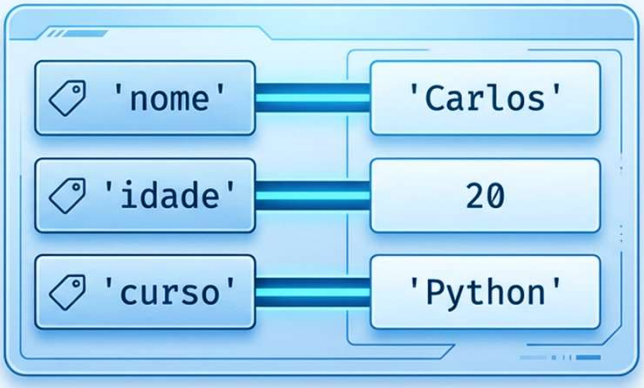
`list(tupla)` → Destrava e vira lista [10, 20, 30]

`tuple(lista)` → Tranca e vira tupla (10, 20, 30)

NotebookLM

Dicionários: O Fichário Inteligente

```
aluno = {
    'nome': 'Carlos',
    'idade': 20,
    'curso': 'Python'
}
```



Esqueça o Índice Zero. Aqui, a busca é por Chave e Valor. Perfeito para registros estruturados.

```
print(aluno['nome'])  # Resultado: Carlos
```

NotebookLM

Matriz de Decisão: Qual Estrutura Usar?

	Lista [ ]	Tupla ( )	Dicionário { }
Sintaxe:	Colchetes: [1, 2, 3]	**Sintaxe:** Parênteses: (1, 2, 3)	**Sintaxe:** Chaves: {'a': 1}
Mutável?:	Sim ✅	**Mutável?:** Não ❌ (Blindada)	**Mutável?:** Sim ✅
Acesso:	Por Índice (0, 1, 2...)	**Acesso:** Por Índice (0, 1, 2...)	**Acesso:** Por Chave (Key)
Uso Ideal:	**Dados dinâmicos** que **crescem** ou **mudam** (ex: carrinho de compras).	**Uso Ideal: Dados fixos de proteção** (ex: coordenadas de um mapa, meses do ano).	**Uso Ideal: Registros detalhados e fichários** (ex: perfil de um usuário).

NotebookLM

Resumo de Bolso (Cheat Sheet)

Criação & Acesso

Criar: `lista = [1, 2, 3]`

Lista mista: `mista = [1, "Dois", True]`

Acesso inicial:
`lista[0]` → primeiro item

Acesso reverso:
`lista[-1]` → último item

Substituir:
`lista[1] = 5`

Modificadores

`lista.append(x)`:
Adiciona "x" no final.

`lista.insert(i, x)`:
Insere "x" no índice "i".

`lista.remove(x)`:
Apaga o primeiro valor "x" que encontrar.

`lista.pop(i)`:
Arranca e retorna o item do índice "i".

Utilitários & Loops

`len(lista)`:
Tamanho total.

`max(lista)`:
Maior valor.

`sum(lista)`:
Soma de tudo.

Loop simples:
`for item in lista:`

Loop indexado:
`for i in range(len(lista)):`

Tire um print desta tela. Mantenha ao lado do seu editor de código. A prática leva à maestria.

NotebookLM

A Jornada do Código

Nível 1
Criar lista e extrair max() e min().

```
lista = [5, 12, 3]
print(max(lista), min(lista))
```

Nível 2
Mostrar elementos na ordem inversa com reverse().

```
lista.reverse()
print(lista)
```

Nível 3
Usar o for para encontrar números pares e calcular médias.

```
pares = [n for n in lista
if n % 2 == 0]
media = sum(pares) / len(pares)
```

Nível 4
Usar .sort() para ordenar e juntar duas listas.

```
lista1.sort()
lista2.sort()
uniao = lista1 + lista2
```

Nível 5 (Boss)
Criar um cadastro de alunos com dicionários e gerar um ranking.

```
alunos = [{'nome': 'Ana', 'nota': 96},
{'nome': 'Leo', 'nota': 85}]

ranking = sorted(alunos, key=lambda
x: x['nota'], reverse=True)
```

Uma estrutura de dados é apenas uma ferramenta. A lógica é sua! Prontos para codar?

NotebookLM